

Objective:

Work with classes, strings, and separate compilation.

Note: This assignment is part 1 of 2. You will need the `Appointment` class to complete the last assignment (homework project 8).

Design a class called *Appointment* that stores the properties of a calendar appointment. Properties include the appointment title, date (**year, month, day**), starting **time (military time)**, and duration. Your class must check for invalid values for year (> 0), month(1-12), day(1-31), time (0-2359), and duration (> 0). Your class must include the following getters and setters for all member data.

Your class must include the following functions (function names and prototypes must match exactly):

- `Appointment();` Default constructor
It should initialize the title to "N/A". The date should be initialized to 1, 1, and 1. The time should be initialized to 0, and duration to 1.
- `Appointment(string appData);`
Add an appointment from a string in the format "title|year|month|day|time|duration". Time is given in standard time and duration is given in minutes. Leading and trailing spaces must be removed.
Example: " Meeting with Bob | 2023 | 6 | 23 | 8:30 aM | 15 "
- `string getTitle();`
- `int getYear();`
- `int getMonth();`
- `int getDay();`
- `int getTime();` //military time
- `int getDuration();`
- `string getDate();` //return a date in the format 2021-11-23
- `string getStandardTime();`
- `void setTitle(string newTitle);`
- `void setYear(int newYear);`
- `void setMonth(int newMonth);`
- `void setDay(int newDay);`
- `void setTime(int newTime);` //military time
- `void setDuration(int newDuration);`
- `void setDate(int newYear, int newMonth, int newDay);`

In addition, the class must include the following helper functions:

- `string militaryToStandard(int time);`
Converts time from military (1830) to standard time ("6:30PM")
- `int standardToMilitary(string time);`
Converts standard time ("8:30PM") to military time (2030). It should handle lower- and upper-case letters.
returns the time in military time format (2030) .

Write a main program to test all the functions above. Make sure you check for invalid values. Separate your program into three files: (appointment.h, appointment.cc, and appointment_main.cc)

Separate Compilation:

First, test your program using a single file "appointment_main.cc". Divide your project into three files:

- appointment.h
 - Contains the class definition
- appointment.cc
 - Contains the class implementation (all the functions)
- appointment_main.cc
 - Main program to test your class

Compiling your project manually:

1. `g++ -Wall -c appointment.cc`
 - This creates the object file appointment.o
2. `g++ -Wall -c appointment_main.cc`
 - This creates the object file appointment_main.o
3. `g++ appointment.o appointment_main.o`
 - This creates the executable a.out

or

```
g++ -Wall -c appointment.cc appointment_main.cc
```

There is a Makefile provided to allow you to compile your program using the command:

```
make
```

You can verify the correctness of your code by running the provided unit tests.

To run the tests:

```
make run_tests
```

You may use any function or library discussed in class or in the chapters we covered from your textbook. Do not use any other libraries or functions.

Hints:

- Write the default constructor, getters, and setters first. Then implement each of the helper functions one at a time.
- Test each function immediately after you write it.
- Write an output function to print all the values in an object.
 - The function `stoi` or `atoi` allows you to convert a string object to an integer.
 - `int x = stoi("123");` will return the integer 123
 - `int x = atoi(s.c_str());` //will return the integer representation of s
- The function `to_string` converts a number to string.
`string s = to_string(123);` will set s to the string "123"
- For the second constructor, use the `find` and the `substr` string member functions.

Grading:

Programs that contain syntax errors will earn zero points.

Programs that do not include functions will also earn zero points.

Programs that use global variables other than constants will earn zero points.

You may use any function or library discussed in class or in the chapters we covered from your textbook. Do not use any other libraries or functions.

Your grade will be determine using the following criteria:

- Correctness (50 points)
 - (5 points) Default constructor
 - (15 points) Second constructor
 - (10 points) Getters and setters
 - (10 points) `militaryToStandard` function
 - (10 points) `standardToMilitary` function
- Style & Documentation (5 points)

Follow the coding style outline on GitHub:

<https://github.com/nasseef/cs/blob/master/docs/coding-style.md>

Submit a link to your repository on Blackboard.